THE DEFINITIVE MASTER REFERENCE HANDBOOK FOR MACHINE LEARNING, DEEP LEARNING & COMPUTER VISION

A Comprehensive One-Stop Pedagogical Guide to 66 Scikit-Learn Algorithms, Mathematical Formulations, Loss Functions, Error Metrics, Computer Vision Architectures, and Unsupervised Learning

Edition 1.0 | Complete Learning & Professional Engineering Manual

Includes Full Mathematical Derivations, Mental Models & Architecture Flowcharts

Contents

List of Abbreviations & Acronyms Index

Abbreviation	Full Term	Brief Description
AIC	Akaike Information Criterion	Model selection score balancing likelihood and parameter count
AUC	Area Under the Curve	Aggregate measure of performance across all classification thresholds
BCE	Binary Cross-Entropy	Log loss metric for binary 0/1 probability classification
BIC	Bayesian Information Criterion	Model selection criterion with stronger penalty for parameters than AIC
CCE	Categorical Cross-Entropy	Multi-class log loss over Softmax probability distributions
CNN	Convolutional Neural Network	Deep network using spatial convolutions for visual grid processing
CV	Cross-Validation	Resampling procedure to evaluate ML model generalization
DBSCAN	Density-Based Spatial Clustering	Clustering algorithm based on dense neighborhood connectivity
DETR	DEtection TRansformer	End-to-end object detector using Transformer encoder-decoder
FCN	Fully Convolutional Network	CNN architecture for pixel-by-pixel semantic segmentation
GMM	Gaussian Mixture Model	Probabilistic soft clustering model assuming Gaussian components
GP	Gaussian Process	Non-parametric Bayesian probabilistic regression framework
IoU	Intersection over Union	Overlap evaluation metric for bounding boxes and masks
KDTree	k-Dimensional Tree	Space-partitioning data structure for fast nearest neighbor search
KL	Kullback-Leibler	Relative entropy measuring divergence between probability distributions
KNN	K-Nearest Neighbors	Instance-based non-parametric classifier/regressor

Abbreviation	Full Term	Brief Description
LARS	Least Angle Regression	Stepwise forward feature selection algorithm
LDA	Linear Discriminant Analysis	Linear supervised classification & dimensionality reduction
LOOCV	Leave-One-Out Cross-Validation	Cross-validation where fold size equals 1 sample
MAE	Mean Absolute Error	Average absolute error distance |y - $\hat{y}$|
MAPE	Mean Absolute Percentage Error	Scale-independent relative error percentage
MCC	Matthews Correlation Coefficient	Balanced correlation coefficient for binary classification
MLP	Multi-Layer Perceptron	Classical feed-forward artificial neural network
MSE	Mean Squared Error	Average squared prediction error distance (y - $\hat{y}$)š
MSLE	Mean Squared Logarithmic Error	Relative logarithmic squared error metric
OLS	Ordinary Least Squares	Linear regression minimizing sum of squared residuals
OMP	Orthogonal Matching Pursuit	Greedy sparse signal approximation algorithm
PCA	Principal Component Analysis	Unsupervised orthogonal variance dimensionality reduction
QDA	Quadratic Discriminant Analysis	Generative classifier with class-specific covariance matrices
RBF	Radial Basis Function	Gaussian kernel measuring Euclidean distance similarity
RMSE	Root Mean Squared Error	Square root of MSE; error in original target units
ROC	Receiver Operating Characteristic	Curve plotting True Positive Rate vs False Positive Rate
RANSAC	RANdom SAmple Consensus	Iterative robust regressor resistant to heavy outliers
SGD	Stochastic Gradient Descent	Iterative optimization using mini-batch sample gradients
sMAPE	Symmetric MAPE	Symmetric relative percentage error metric bounded [0%, 200%]
SMO	Sequential Minimal Optimization	Algorithm for fast quadratic optimization in SVMs
SSD	Single Shot MultiBox Detector	Real-time one-stage object detection CNN
SVC / SVR	Support Vector Classifier / Regressor	Maximum-margin hyperplane support vector machine
t-SNE	t-Distributed Stochastic Neighbor Embedding	Non-linear manifold visualization technique
UMAP	Uniform Manifold Approximation & Projection	Fast non-linear manifold dimensionality reduction
ViT	Vision Transformer	Transformer architecture applied directly to image patches
WAPE	Weighted Absolute Percentage Error	Volume-weighted absolute percentage error metric
YOLO	You Only Look Once	Real-time single-stage object detector

Chapter 1: Foundational Framework of Model Evaluation & Error Metrics

1.1 Confusion Matrix Layout & Formulations

	Actual Positive	**Actual Negative**
Predicted Positive	TP (True Positive) Correctly identified positive class	FP (False Positive) Type I Error - False Alarm
Predicted Negative	FN (False Negative) Type II Error - Missed Case	TN (True Negative) Correctly identified negative class

1.2 Situation-to-Metric Selection Guide

Business / Modeling Situation	**Recommended Metric**	**Key Reason / Focus**
Classes are balanced	Accuracy	Overall ratio of correct predictions
False positives are expensive	Precision	Minimizes false alarms (e.g., spam filter)
False negatives are expensive	Recall / Sensitivity	Minimizes missed critical cases (e.g., medical diagnosis)
Need balance of Precision + Recall	F1 Score	Harmonic mean balancing FP and FN penalties
Very imbalanced dataset	F1 / MCC / PR-AUC	Avoids accuracy inflation on majority class
Need probability quality	Log Loss	Penalizes confident incorrect probability predictions
Need threshold-independent ranking	ROC-AUC	Measures class separation ability across all thresholds
Medical diagnosis	Recall + Specificity + ROC-AUC	Ensures high detection while tracking true negatives
Image/object classification	Accuracy + F1 + Confusion Matrix	Comprehensively tracks per-class errors
Multi-class classification	Macro F1 + Weighted F1 + Accuracy	Balances overall performance and minority classes

1.3 Classification Loss Functions & Metrics Summary

Function Name	**Full Name / Variant**	**Main Purpose**	**Key Property & Formula Note**
BCE	Binary Cross Entropy	Binary classification	Penalizes wrong probability outputs for 0/1 target
BCE with Logits	Binary CE with Logits	Binary classification directly from raw logits	More numerically stable (combines Sigmoid + BCE)
Categorical CE	Categorical Cross Entropy	Multi-class classification	Requires one-hot encoded targets with Softmax
Sparse Categorical CE	Sparse Categorical CE	Multi-class classification	Accepts integer class labels directly (memory efficient)
Focal Loss	Focal Loss	Handles severe class imbalance	Down-weights easy examples with factor (1-p)ˆ
Hinge Loss	Hinge Loss	SVM-style classification	Maximizes margin max(0, 1 - yůf(x)) for y {-1, +1}

Function Name	Full Name / Variant	Main Purpose	Key Property & Formula Note
Squared Hinge Loss	Squared Hinge Loss	SVM-style classification	Quadratic penalty for margin violations [max(0, 1-yůf(x))]š
KL Divergence	Kullback-Leibler Divergence	Probability distribution fitting	Measures divergence between predicted Q and true P
Label Smoothing Loss	Label Smoothing CE	Prevents overconfident predictions	Softens hard 0/1 targets into 0.05/0.95 distributions

1.4 Regression Loss Functions & Evaluation Metrics Summary

Metric / Loss	Full Name	Range	Best	Main Purpose & Key Property
MAE	Mean Absolute Error	0–	0	Average absolute error. Robust to outliers.
MedAE	Median Absolute Error	0–	0	Median prediction error. Highly robust against extreme outliers.
MSE	Mean Squared Error	0–	0	Penalizes large errors strongly (squared error).
RMSE	Root Mean Squared Error	0–	0	Square root of MSE; error is in same units as target.
Rš	R-squared (Coeff. of Det.)	- to 1	1	Proportion of target variance explained by model.
Adjusted Rš	Adjusted R-squared	- to 1	1	Rš penalized for adding non-informative feature variables.
Explained Var.	Explained Variance Score	- to 1	1	Measures variation captured (ignores mean bias).
MAPE	Mean Absolute % Error	0–%	0%	Percentage-based error. Problematic when actual y 0.
sMAPE	Symmetric MAPE	0–200%	0%	Symmetric percentage error; less sensitive to zero target issues.
MSPE	Mean Squared % Error	0–	0	Squared percentage error for severe relative penalty.
MSLE	Mean Squared Log Error	0–	0	Penalizes relative log-scale error; useful for skewed targets.

Metric / Loss	Full Name	Range	Best	Main Purpose & Key Property
RMSLE	Root Mean Squared Log Error	0–	0	Root version of MSLE; reduces impact of large target scale.
Huber Loss	Huber / Smooth L1 Loss	0–	0	Combines MSE (small errors) + MAE (large errors). Outlier-robust.
Log-Cosh Loss	Logarithm of Hyperbolic Cosine	0–	0	Smooth robust loss, twice-differentiable everywhere.
Quantile Loss	Pinball Loss	0–	0	Predicts specified percentiles (used for prediction intervals).
Max Error	Maximum Error	0–	0	Captures the worst single prediction error in dataset.

Chapter 2: Classification Models & Architectures (30 Models)

1. LinearSVC

Mental Model

Find a linear hyperplane that separates classes with the largest possible geometric margin.

Mathematical Formulation & Prediction

f(x) = wˆT * x + b

Prediction: y_hat = +1 if f(x) > 0 else -1

Optimization & Loss Function

min_{w,b} 0.5 * ||w||š + C * max(0, 1 - y_i * (wˆT * x_i + b))š

The loss term is Squared Hinge Loss (or standard Hinge Loss).

Meaning Breakdown & Mental Flowchart

||w||š

Keep boundary simple / maximize geometric margin

hinge loss

Punish incorrectly classified or insufficiently separated samples

C hyperparameter

Trade-off between wide margin and classification error

2. SGDClassifier

Mental Model

Take a small sample or single data point, calculate the prediction error, and instantly update weights via gradient step.

Mathematical Formulation & Prediction

General Objective: J(w) = (1/n) * L(y_i, f(x_i)) + * R(w)

Single Step Update: w_{t+1} = w_t - * J(w_t)

Optimization & Loss Function

Configurable loss L(y, f(x)):
- L = Hinge (SVM)
- L = Log loss (Logistic Regression)
- L = Modified Huber (probabilistic SVM)

Regularization R(w): L2 norm 0.5*||w||š, L1 norm ||w||_1, or ElasticNet.

Meaning Breakdown & Mental Flowchart

prediction

Make quick output guess for current sample

calculate error

Compute loss value L(y_i, y_hat_i)

calculate gradient

Compute gradient J with respect to weight vector w

change weights

Update weights w_{t+1} = w_t - * J and repeat

3. MLPClassifier

Mental Model

Stack multiple layers of artificial neurons to learn complex, non-linear feature representations automatically.

Mathematical Formulation & Prediction

Single Neuron: z = wˆT * x + b, Activation: a = f(z)

Network Layers: h_1 = f(W_1 * x + b_1), h_2 = f(W_2 * h_1 + b_2), y_hat = Softmax(W_3 * h_2 + b_3)

Softmax Probability: P(y=k|x) = exp(z_k) / _j exp(z_j)

Optimization & Loss Function

Cross-Entropy Loss: L = - y_k * log(y_hat_k) + (/ 2n) * ||W_l||_Fš

Training uses Backpropagation + Gradient Descent (Adam, L-BFGS, or SGD).

Meaning Breakdown & Mental Flowchart

input layer

Receive raw input feature vector x

hidden layers

Extract non-linear hierarchical feature embeddings

softmax output

Produce normalized probability distribution across classes

backprop

Propagate gradient error backwards to update weight matrices W_l

4. Perceptron

Mental Model

A single linear neuron that tests predictions and shifts its boundary weights only when it makes a mistake.

Mathematical Formulation & Prediction

Prediction: y_hat = sgn(wˆT * x + b)

Condition to update: If y_i * (wˆT * x_i + b) 0 (Misclassified)

Optimization & Loss Function

Update Rule:
w w + * y_i * x_i
b b + * y_i

Meaning Breakdown & Mental Flowchart

correct prediction

Do nothing (weights stay unchanged)

misclassified sample

Shift weight vector w in the direction of the error y_i * x_i

simple loop

Iterate until linear convergence or max iterations reached

5. LogisticRegression

Mental Model

Compute a linear score for a sample, then map it through a Sigmoid curve to output a clean class probability between 0 and 1.

Mathematical Formulation & Prediction

Linear Score: z = wˆT * x + b

Sigmoid Function: P(y=1|x) = (z) = 1 / (1 + exp(-z))

Optimization & Loss Function

Binary Cross-Entropy Loss:
L(w) = - (1/n) * [y_i * log(p_i) + (1 - y_i) * log(1 - p_i)] + (1 / 2C) * ||w||š

Optimized via L-BFGS, Newton-CG, or Liblinear.

Meaning Breakdown & Mental Flowchart

linear score z

Combine features linearly z = wˆT * x + b

sigmoid (z)

Squash z to probability scale [0, 1]

cross-entropy

Penalize confident wrong probability guesses exponentially

gradient descent

Adjust w to maximize log-likelihood of dataset

6. LogisticRegressionCV

Mental Model

Fit Logistic Regression across multiple internal cross-validation folds to automatically discover the best regularization strength C.

Mathematical Formulation & Prediction

P(y=1|x) = (wˆT * x + b)

Validation Score: CV_Score(C_k) = (1/K) * _{v=1}ˆK Accuracy_val(C_k, Fold_v)

Optimization & Loss Function

Solves internal grid search over a grid of C values:
C^* = argmax_{C_k} CV_Score(C_k)

Final fit uses C^* on the full dataset.

Meaning Breakdown & Mental Flowchart

C grid search

Test multiple regularization candidate values C

K-fold split

Evaluate out-of-fold cross-validation accuracy for each C

select optimal C*

Pick C* with maximum validation score

final model

Fit Logistic Regression using optimal C*

7. SVC (Support Vector Classifier)

Mental Model

Project data into higher-dimensional space using kernel functions where classes become linearly separable by a maximum-margin hyperplane.

Mathematical Formulation & Prediction

Dual Decision Function: f(x) = sgn(_{i SV} _i * y_i * K(x_i, x) + b)

Kernels K(x_i, x):
- Linear: x_i^T * x
- RBF: exp(- * ||x_i - x||š)
- Poly: (* x_i^T * x + r)^d

Optimization & Loss Function

Dual Quadratic Optimization:
max_ _i - 0.5 * _i _j _i * _j * y_i * y_j * K(x_i, x_j) s.t. 0 _i C, _i * y_i = 0

Solved using Sequential Minimal Optimization (SMO).

Meaning Breakdown & Mental Flowchart

kernel K(x, x_i)

Compute non-linear similarity between samples in feature space

support vectors

Identify key boundary points where _i > 0

max margin boundary

Construct optimal separating boundary between classes

8. CalibratedClassifierCV

Mental Model

Wrap an uncalibrated classifier (like SVM or Naive Bayes) and pass its scores through a secondary mapping to output accurate, reliable probabilities.

Mathematical Formulation & Prediction

Platt Scaling (Sigmoid): P(y=1|f) = 1 / (1 + exp(A * f(x) + B))

Isotonic Regression: P(y=1|f) = g(f(x)) where g is non-decreasing step function

Optimization & Loss Function

Fits parameters A, B via Maximum Likelihood on validation fold scores f(x), or fits isotonic step curve g to minimize squared probability error.

Meaning Breakdown & Mental Flowchart

base model f(x)

Get uncalibrated confidence score / distance f(x)

calibration map

Pass f(x) through Sigmoid curve (Platt) or Monotonic step (Isotonic)

calibrated P(y|x)

Output true calibrated probability reflecting real-world frequencies

9. PassiveAggressiveClassifier

Mental Model

Be passive (do nothing) if a sample is correctly classified with margin; become aggressive (make huge step update) if error occurs.

Mathematical Formulation & Prediction

Hinge Loss: L_t = max(0, 1 - y_t * (w_tˆT * x_t))

Weight Update: w_{t+1} = w_t + _t * y_t * x_t

Optimization & Loss Function

Step Size Tau: _t = min(C, L_t / ||x_t||š)

Optimizes constrained distance minimization ||w - w_t||š subject to linear loss constraint.

Meaning Breakdown & Mental Flowchart

sample correct (L_t=0)

Passive mode: Keep weights w_{t+1} = w_t

sample error (L_t>0)

Aggressive mode: Update weights proportional to loss L_t

C bound

Clip step size _t by C to prevent noisy over-corrections

10. LabelPropagation

Mental Model

Build a similarity graph of all data points and let known class labels flow along graph edges to label unlabeled points.

Mathematical Formulation & Prediction

Graph Weight: W_{ij} = exp(- * ||x_i - x_j||š)

Label Transition: Y_{t+1} = Dˆ{-1} * W * Y_t

Optimization & Loss Function

Iteratively multiplies probability label matrix Y by normalized transition matrix Dˆ{-1}W, then clamps (resets) known labeled nodes back to their true values.

Meaning Breakdown & Mental Flowchart

similarity graph

Connect data points with weights based on distance RBF kernel

label diffusion

Propagate label distribution from labeled to unlabeled nodes

hard clamping

Reset original ground-truth labels after each iteration until convergence

11. LabelSpreading

Mental Model

Propagate labels smoothly over a normalized graph Laplacian while allowing soft noise adjustments on original labels.

Mathematical Formulation & Prediction

Symmetric Normalized Graph: S = Dˆ{-1/2} * W * Dˆ{-1/2}

Iterative Update: Y_{t+1} = * S * Y_t + (1 -) * Y_0

Optimization & Loss Function

Closed-Form Solution:
Yˆ* = (1 -) * (I - * S)ˆ{-1} * Y_0

parameter balances graph smoothness versus initial label retention.

Meaning Breakdown & Mental Flowchart

normalized graph S

Scale graph weights symmetrically to avoid degree bias

soft clamping

Blend graph consensus () with original labels (1-)

closed-form Y*

Solve exact steady-state label equilibrium matrix directly

12. RandomForestClassifier

Mental Model

Grow a forest of diverse decision trees on random data subsets and random feature splits, then combine their votes.

Mathematical Formulation & Prediction

Tree Vote: P_m(y=c|x) from m-th tree trained on Bootstrap Sample B_m

Forest Output: P(y=c|x) = (1/M) * _{m=1}^M P_m(y=c|x)

Optimization & Loss Function

Each tree node selects optimal split by sampling a random feature subset of size max_features (D) and maximizing Gini Gain I_G = I_G(parent) - [(N_L/N) I_G(L) + (N_R/N) I_G(R)].

Meaning Breakdown & Mental Flowchart

bootstrap sampling

Train each tree on a random sample with replacement

feature randomization

At each split node, search only a random subset of features

deep trees

Grow unpruned trees to capture complex interactions

majority vote

Average predictions to reduce variance drastically

13. GradientBoostingClassifier

Mental Model

Build decision trees sequentially, where each new tree focuses on correcting the specific errors (pseudo-residuals) of the previous ensemble.

Mathematical Formulation & Prediction

Ensemble Prediction: F_m(x) = F_{m-1}(x) + * _m * h_m(x)

Pseudo-Residual: r_{i,m} = - [L(y_i, F(x_i)) / F(x_i)]_{F=F_{m-1}}

Optimization & Loss Function

Fits tree h_m(x) to negative loss gradient r_{i,m} using log-loss deviance, then scales by learning rate .

Meaning Breakdown & Mental Flowchart

current model F_{m-1}

Generate predictions for all training samples

compute residuals r_m

Calculate negative gradient direction of loss function

fit new tree h_m

Train tree to predict error residuals r_m

add with shrinkage

Update F_m(x) = F_{m-1}(x) + * h_m(x) to prevent overfitting

14. QuadraticDiscriminantAnalysis

Mental Model

Model each class as a separate multivariate Gaussian distribution with its own unique covariance matrix, creating quadratic decision boundaries.

Mathematical Formulation & Prediction

Discriminant Score:
_k(x) = -0.5 * log|_k| - 0.5 * (x - _k)^T * _k^{-1} * (x - _k) + log(P(Y=k))

Optimization & Loss Function

Maximum Likelihood Estimation:
Estimates class means _k, class covariance matrices _k, and prior probabilities P(Y=k) directly from data.

Meaning Breakdown & Mental Flowchart

class mean _k

Find average center location of class k

covariance _k

Fit class-specific shape, spread, and orientation matrix

quadratic boundary

Compare quadratic distance scores and pick max _k(x)

15. HistGradientBoostingClassifier

Mental Model

Discretize continuous features into integer bins to compute gradient histograms instantly, speeding up gradient boosting exponentially.

Mathematical Formulation & Prediction

Binning Transformation: x_{binned} = QuantileBin(x, max_bins=255)

Histogram Gradient: G_bin = _{i Bin} g_i, H_bin = _{i Bin} h_i

Optimization & Loss Function

Evaluates split thresholds over 256 discrete bins instead of sorting all individual sample values, reducing complexity from O(N log N) to O(max_bins).

Meaning Breakdown & Mental Flowchart

255 quantile bins

Group continuous numbers into 256 integer buckets

gradient histogram

Sum first (g_i) and second (h_i) derivatives per bin

instant split search

Scan 256 bin boundaries to find best split threshold

super fast boosting

Train gradient boosted trees 10x-100x faster

16. RidgeClassifierCV

Mental Model

Cast classification as a multi-target linear regression with L2 regularization and tune parameter via fast analytical Leave-One-Out Cross-Validation.

Mathematical Formulation & Prediction

Target Encoding: Y {-1, +1}ˆ{N Œ C}

Prediction: y_hat = argmax_c (w_cˆT * x + b_c)

Optimization & Loss Function

Analytical LOOCV Error:
ˆ* = argmin_{_k} (1/n) * [(y_i - y_hat_{-i}(_k))š]

Computed instantly via SVD matrix decomposition without retraining.

Meaning Breakdown & Mental Flowchart

label matrix Y

Convert labels to -1/+1 multi-column indicators

SVD decomposition

Decompose feature matrix X = U VˆT once

instant LOOCV score

Evaluate validation error for all values simultaneously

select best *

Pick optimal * and compute final classifier weights

17. RidgeClassifier

Mental Model

Convert classification target labels to binary indicator numbers, fit a Ridge regression model, and pick the class with the highest score.

Mathematical Formulation & Prediction

Indicator Matrix: Y {-1, +1}ˆ{N Œ C}

Weight Solution: w = (XˆT * X + * I)ˆ{-1} * XˆT * Y

Optimization & Loss Function

Prediction Rule:
y_hat = argmax_c (w_cˆT * x + b_c)

Minimizes squared indicator error with L2 penalty.

Meaning Breakdown & Mental Flowchart

convert targets

Map class labels to -1/+1 columns

closed-form matrix

Solve w = (XˆT X + I)ˆ{-1} XˆT Y in one step

argmax decision

Output class corresponding to highest continuous linear output

18. AdaBoostClassifier

Mental Model

Combine weak decision stumps sequentially by boosting weights of misclassified samples after each round.

Mathematical Formulation & Prediction

Ensemble Output: H(x) = sgn(_{m=1}ˆM _m * h_m(x))

Learner Weight: _m = 0.5 * log((1 - e_m) / e_m)

Optimization & Loss Function

Sample Weight Update:
w_{i, m+1} = w_{i, m} * exp(- _m * y_i * h_m(x_i)), normalized so w_i = 1.

Implicitly minimizes Exponential Loss L(y, f(x)) = exp(-y * f(x)).

Meaning Breakdown & Mental Flowchart

train weak stump

Fit simple 1-split tree h_m on weighted data

calculate error e_m

Compute weighted error rate of weak learner

compute _m

Assign voting weight _m to current stump based on accuracy

re-weight samples

Increase weights of misclassified data points for next stump

19. ExtraTreesClassifier

Mental Model

Build a forest of trees using random split thresholds for candidate features to maximize randomness and lower model variance.

Mathematical Formulation & Prediction

Split Threshold: t_j ~ Uniform(min(x_j), max(x_j)) across random feature subset

Optimization & Loss Function

Computes impurity gain for randomly drawn thresholds instead of searching for the exact mathematical optimum split point, reducing variance and computation time.

Meaning Breakdown & Mental Flowchart

random features

Select random subset of features at each node

random thresholds

Draw random split threshold t_j uniformly within range

pick best random split

Choose the best among the random candidates

forest voting

Average votes across trees to lower prediction variance

20. KNeighborsClassifier

Mental Model

Store all training data and classify a new point based on the majority vote of its K closest neighbors in feature space.

Mathematical Formulation & Prediction

Distance Metric: d(x, z) = (|x_j - z_j|^p)^{1/p}

Class Probability: P(Y=c|x) = (1/K) * _{i N_K(x)} I(y_i = c)

Optimization & Loss Function

Lazy Learning (No explicit training phase).
Search Acceleration: Uses KDTree or BallTree index structures to find nearest points fast.

Meaning Breakdown & Mental Flowchart

store samples

Keep training vectors in memory (or tree index)

find K neighbors

Compute distance to all points and select K closest

vote / **average**

Count class labels among K neighbors and predict majority class

21. BaggingClassifier

Mental Model

Train multiple copies of a base model on random bootstrap subsets of data and average their predictions to reduce model variance.

Mathematical Formulation & Prediction

Bootstrap Samples: B_1, B_2, ..., B_M drawn with replacement from dataset X

Ensemble Output: f_{bag}(x) = Mode(h_1(x), h_2(x), ..., h_M(x))

Optimization & Loss Function

Fits base estimator h_m independently on each bootstrap subset B_m.

Meaning Breakdown & Mental Flowchart

create M subsets

Draw M bootstrap random samples with replacement

fit M base models

Train independent base model on each subset

combine predictions

Take majority vote / average probability across models

22. BernoulliNB

Mental Model

Calculate class probabilities assuming binary boolean features (0 or 1) that are independent given the class label.

Mathematical Formulation & Prediction

Likelihood: P(x|Y=c) = _{j=1}^D [p_{cj}^{x_j} * (1 - p_{cj})^{(1 - x_j)}]

Probability Parameter: p_{cj} = (N_{c,j} +) / (N_c + 2)

Optimization & Loss Function

Decision Rule: y_hat = argmax_c [log P(Y=c) + log P(x_j|Y=c)]

Uses Laplace smoothing to prevent zero probability issues.

Meaning Breakdown & Mental Flowchart

binarize features

Check feature presence (x_j=1) or absence (x_j=0)

count frequencies

Calculate class probability p_{cj} for each feature

naive bayes rule

Multiply probabilities assuming conditional independence

23. LinearDiscriminantAnalysis

Mental Model

Project data onto a lower-dimensional axis that maximizes class separation relative to within-class variance, assuming shared covariance.

Mathematical Formulation & Prediction

Linear Discriminant Function:
_k(x) = x^T * ^{-1} * _k - 0.5 * _k^T * ^{-1} * _k + log(P(Y=k))

Optimization & Loss Function

Assumes shared covariance matrix across all classes, resulting in linear decision hyperplanes w_k^T * x + b_k.

Meaning Breakdown & Mental Flowchart

shared covariance

Pool feature variances across all classes

calculate means _k

Find average feature vector per class

linear decision surface

Separate classes with linear hyperplane boundaries

24. GaussianNB

Mental Model

Assume continuous features within each class follow bell-shaped Gaussian distributions and combine their probabilities independently.

Mathematical Formulation & Prediction

Gaussian Likelihood:
P(x_j | Y=c) = (1 / (2 * _{cj}š)) * exp(- (x_j - _{cj})š / (2 * _{cj}š))

Optimization & Loss Function

Estimates mean _{cj} and variance _{cj}š for each feature j per class c using Maximum Likelihood.

Meaning Breakdown & Mental Flowchart

fit gaussian curve

Compute mean and variance š per feature per class

compute probability

Evaluate Gaussian likelihood P(x_j|c) for input value

bayes rule decision

Multiply feature likelihoods with class priors P(c)

25. NuSVC

Mental Model

Control SVM margin errors directly using parameter (nu) which bounds the fraction of support vectors and margin violations.

Mathematical Formulation & Prediction

Primal Optimization:
min_{w,b,,} 0.5 * ||w||š - * + (1/n) * _i s.t. y_i * (wˆT (x_i) + b) - _i

Optimization & Loss Function

Parameter (0, 1] automatically sets an upper bound on margin error fraction and lower bound on support vector count.

Meaning Breakdown & Mental Flowchart

nu parameter

Set target percentage of allowed margin errors

flexible margin

Dynamically adjust margin width parameter

SMO solver

Solve dual QP formulation via LIBSVM

26. DecisionTreeClassifier

Mental Model

Ask a series of binary yes/no questions on features to recursively split data into increasingly pure class regions.

Mathematical Formulation & Prediction

Node Impurity I(A):
- Gini: 1 - p_kš
- Entropy: - p_k * log2(p_k)

Information Gain: I = I(Parent) - [(N_L/N) I(L) + (N_R/N) I(R)]

Optimization & Loss Function

Finds feature j and split threshold t maximizing Information Gain I recursively until stopping criteria are met.

Meaning Breakdown & Mental Flowchart

evaluate all splits

Test feature thresholds to find maximum purity increase

split node

Divide data into left and right child nodes

repeat recursively

Continue splitting until leaf nodes are pure or max depth reached

27. NearestCentroid

Mental Model

Represent each class by its mean center vector (centroid) and assign new samples to the closest centroid.

Mathematical Formulation & Prediction

Class Centroid: _c = (1 / N_c) * _{i: y_i = c} x_i

Prediction: y_hat = argmin_c d(x, _c)

Optimization & Loss Function

Calculates Euclidean or Manhattan distance to each class mean vector _c.

Meaning Breakdown & Mental Flowchart

calculate centroids

Compute average center point _c for each class

measure distance

Calculate distance from sample x to all centroids

assign class

Pick class label of the closest centroid

28. ExtraTreeClassifier

Mental Model

Build a single extremely randomized decision tree where candidate split thresholds are drawn completely at random.

Mathematical Formulation & Prediction

Split Threshold: t_j ~ Uniform(min(x_j), max(x_j))

Optimization & Loss Function

Selects random split thresholds for candidate features and picks the best among them, speeding up tree construction.

Meaning Breakdown & Mental Flowchart

draw random t_j

Pick random split threshold for candidate features

evaluate impurity

Compute Gini impurity for random threshold splits

split & repeat

Split data fast with lower computational overhead

29. CheckingClassifier

Mental Model

An internal diagnostic test model used to check Scikit-Learn estimator API compliance and data input formatting.

Mathematical Formulation & Prediction

Internal API Check: Validate X shape, y format, and fit/predict method signatures.

Optimization & Loss Function

Performs no real ML training; verifies estimator contract during automated testing.

Meaning Breakdown & Mental Flowchart

receive input X, y

Accept data arrays

validate contract

Check API compatibility and parameter constraints

return mock output

Provide mock test response

30. DummyClassifier

Mental Model

Make simple non-smart baseline predictions (like always predicting the majority class) to benchmark real ML model uplift.

Mathematical Formulation & Prediction

Strategy 'most_frequent': P(y=c|x) = 1 if c = Mode(Y) else 0

Strategy 'stratified': P(y=c|x) = Prior(c) = N_c / N

Optimization & Loss Function

Computes simple summary statistics on target labels Y without looking at input features X.

Meaning Breakdown & Mental Flowchart

ignore features X

Do not inspect input feature values

calculate label stat

Count class frequencies in training targets Y

predict baseline

Output majority class or random prior distribution

Chapter 3: Regression Models & Architectures (36 Models)

31. SVR

Mental Model

Fit a prediction tube around data points where errors within threshold are completely ignored, penalizing only points outside.

Mathematical Formulation & Prediction

Dual Function: f(x) = _{i SV} (_i - _iˆ*) * K(x_i, x) + b

-Insensitive Loss: L_(y, f(x)) = max(0, |y - f(x)| -)

Optimization & Loss Function

min_{w,b,,ˆ*} 0.5 * ||w||š + C * (_i + _iˆ*)
s.t. |y_i - (wˆT (x_i) + b)| + _i

Meaning Breakdown & Mental Flowchart

insensitive tube

Ignore small errors falling within distance of tube

slack variables _i

Penalize points falling outside the -tube linearly

C parameter

Control trade-off between tube smoothness and error penalty

32. RandomForestRegressor

Mental Model

Grow multiple regression trees on random data samples and average their continuous predictions to reduce model variance.

Mathematical Formulation & Prediction

Tree Output: T_m(x) for m-th regression tree

Ensemble Output: f(x) = (1/M) * _{m=1}ˆM T_m(x)

Optimization & Loss Function

Node Split Criterion: Minimizes Variance / MSE
Split Loss = _{i Left} (y_i - c_1)š + _{i Right} (y_i - c_2)š

Meaning Breakdown & Mental Flowchart

bootstrap samples

Train trees on independent random data subsets

feature subsets

Select random feature candidate subsets at each split node

average predictions

Average tree outputs to produce smooth continuous predictions

33. ExtraTreesRegressor

Mental Model

Average predictions across regression trees built with completely random split thresholds for candidate features.

Mathematical Formulation & Prediction

Split Threshold: t_j ~ Uniform(min(x_j), max(x_j))

Ensemble Prediction: f(x) = (1/M) * _{m=1}^M T_m(x)

Optimization & Loss Function

Evaluates random split thresholds to reduce tree correlation and overall prediction variance.

Meaning Breakdown & Mental Flowchart

random thresholds

Draw random split threshold t_j per candidate feature

fit extra trees

Build highly randomized regression trees

average ensemble

Average predictions for robust, low-variance regression

34. AdaBoostRegressor

Mental Model

Train sequence of weak regression trees, increasing weights of samples with large relative error after each stage.

Mathematical Formulation & Prediction

Relative Error: e_{im} = |y_i - T_m(x_i)| / Max_Error

Ensemble Prediction: Weighted Median of {T_m(x)} using weights log(1/_m)

Optimization & Loss Function

Updates sample weights w_{i, m+1} = w_{i, m} * _m^{(1 - e_{im})} where _m = e_m / (1 - e_m).

Meaning Breakdown & Mental Flowchart

fit weak tree

Train regression tree T_m on weighted sample data

calculate errors

Compute relative error e_{im} for each sample

weighted median

Combine tree outputs using weighted median prediction

35. NuSVR

Mental Model

Automatically control the width of the insensitive error tube using parameter (nu).

Mathematical Formulation & Prediction

Objective:
min_{w,b,,ˆ*,} 0.5 * ||w||š + C * (* + (1/n) * (_i + _iˆ*))

s.t. |y_i - (wˆT (x_i) + b)| + _i

Optimization & Loss Function

Parameter (0, 1] bounds the fraction of support vectors and automatically determines tube width .

Meaning Breakdown & Mental Flowchart

nu parameter

Set desired fraction of support vector data points

auto tube width

Automatically compute optimal insensitive tube margin

SMO optimization

Solve kernelized dual SVR problem via LIBSVM

36. GradientBoostingRegressor

Mental Model

Fit decision trees sequentially to the residual errors (gradients) of the existing ensemble.

Mathematical Formulation & Prediction

Ensemble Update: F_m(x) = F_{m-1}(x) + * _m * h_m(x)

Gradient Residual: r_{im} = - [L(y_i, F(x_i)) / F(x_i)]_{F=F_{m-1}}

Optimization & Loss Function

Loss Functions: 'squared_error' (MSE), 'absolute_error' (MAE), 'huber', 'quantile'.

Meaning Breakdown & Mental Flowchart

calculate residuals

Compute error r_im = y_i - F_{m-1}(x_i)

fit new tree h_m

Train tree h_m to predict residuals r_im

apply shrinkage

Scale tree output by learning rate and add to ensemble

37. KNeighborsRegressor

Mental Model

Predict a continuous target by averaging the target values of the K nearest neighbors in feature space.

Mathematical Formulation & Prediction

Prediction: f(x) = _{i N_K(x)} w_i * y_i / w_i

w_i = 1 (Uniform) or w_i = 1 / d(x, x_i) (Distance-weighted)

Optimization & Loss Function

Uses Minkowski distance d(x, z) = (|x_j - z_j|^p)^{1/p} with KDTree/BallTree search.

Meaning Breakdown & Mental Flowchart

find K neighbors

Locate K nearest training samples by distance

weight targets

Assign weights w_i based on inverse distance

weighted average

Compute weighted target mean to produce prediction f(x)

38. HistGradientBoostingRegressor

Mental Model

Discretize continuous inputs into integer bins to compute gradient histograms rapidly for fast regression tree splits.

Mathematical Formulation & Prediction

Binning: x_{binned} = QuantileBin(x, max_bins=255)

Tree Split: Finds bin boundary maximizing loss reduction using gradient/hessian sums per bin.

Optimization & Loss Function

Accelerates gradient boosting on large datasets while natively supporting missing values.

Meaning Breakdown & Mental Flowchart

bin features

Group continuous feature values into 256 discrete bins

sum gradients

Accumulate gradient & hessian sums per bin

rapid split search

Scan 256 bin boundaries for optimal split threshold

39. BaggingRegressor

Mental Model

Average continuous outputs from multiple base regressors trained on random bootstrap samples.

Mathematical Formulation & Prediction

Bootstrap Samples: B_1, B_2, ..., B_M

Prediction: f_{bag}(x) = (1/M) * _{m=1}^M h_m(x)

Optimization & Loss Function

Fits base regressor h_m independently on each random bootstrap data subset.

Meaning Breakdown & Mental Flowchart

create M samples

Draw bootstrap random data subsets with replacement

fit regressors

Train independent base regressor on each subset

average outputs

Average predicted values across all M base regressors

40. MLPRegressor

Mental Model

Pass features through hidden neural layers and output a continuous prediction via a linear output neuron.

Mathematical Formulation & Prediction

Hidden Layers: h_1 = f(W_1 * x + b_1), h_2 = f(W_2 * h_1 + b_2)

Linear Output: f(x) = W_3 * h_2 + b_3

Optimization & Loss Function

MSE Loss: L = 0.5 * (1/n) * (y_i - f(x_i))š + (/ 2n) * ||W_l||_Fš

Optimized via Backpropagation with Adam or L-BFGS.

Meaning Breakdown & Mental Flowchart

hidden activations

Compute non-linear feature representations in hidden layers

linear output neuron

Produce unconstrained continuous target value f(x)

MSE loss backprop

Compute squared error and backpropagate weight updates

41. HuberRegressor

Mental Model

Fit a robust linear model that uses squared loss for small errors and linear loss for large outlier errors.

Mathematical Formulation & Prediction

Huber Loss Function H_(z):
- 0.5 * zš if |z|
- * |z| - 0.5 * š if |z| >

Optimization & Loss Function

min_{w, } [+ H_((y_i - wˆT x_i) /) *] + * ||w||š_2

Optimizes weights w and scale parameter simultaneously.

Meaning Breakdown & Mental Flowchart

small errors (|z|)

Behave like MSE (quadratic smooth loss)

large errors (|z|>)

Behave like MAE (linear loss; immune to outlier blowup)

scale parameter

Scale residuals dynamically to identify outlier threshold

42. LinearSVR

Mental Model

Fit a linear prediction function using -insensitive loss solved rapidly via LIBLINEAR.

Mathematical Formulation & Prediction

Prediction: f(x) = wˆT * x + b

Loss: L_ = max(0, |y_i - (wˆT x_i + b)| -)

Optimization & Loss Function

min_{w,b} 0.5 * ||w||š + C * max(0, |y_i - (wˆT x_i + b)| -)

Fast linear solver using Coordinate Descent.

Meaning Breakdown & Mental Flowchart

linear prediction

Calculate linear prediction wˆT * x + b

-insensitive loss

Ignore errors inside -tube margin

LIBLINEAR solver

Optimize linear weights fast for large datasets

43. RidgeCV

Mental Model

Fit Ridge regression and select the optimal L2 penalty using efficient analytical Leave-One-Out Cross-Validation.

Mathematical Formulation & Prediction

Analytical LOOCV MSE:
LOOCV_Error(_k) = (1/n) * [(y_i - y_hat_{-i}(_k))š]

Optimization & Loss Function

Computes LOOCV error for all values simultaneously using SVD decomposition without refitting models.

Meaning Breakdown & Mental Flowchart

SVD decomposition

Decompose feature matrix X once via SVD

analytical LOOCV

Compute leave-one-out validation error for all candidates

select optimal *

Pick * with minimal validation MSE and return final weights

44. BayesianRidge

Mental Model

Estimate linear regression parameters probabilistically under Gaussian priors, providing prediction uncertainty.

Mathematical Formulation & Prediction

Posterior Distribution: P(w|y, X) ~ N(_N, _N)

_N = * _N * XˆT * y, _N = (* I + * XˆT * X)ˆ{-1}

Optimization & Loss Function

Estimates precision hyperparameters (noise precision) and (weight prior precision) via Empirical Bayes (Type-II ML).

Meaning Breakdown & Mental Flowchart

gaussian prior

Assume Gaussian weight prior P(w|) = N(0, ˆ{-1} I)

posterior _N, _N

Calculate exact posterior weight mean and covariance

prediction variance

Output expected prediction mean and uncertainty variance

45. Ridge

Mental Model

Shrink linear regression coefficients towards zero using an L2 penalty to prevent overfitting and handle multicollinearity.

Mathematical Formulation & Prediction

Objective: min_{w} ||y - X w||š_2 + * ||w||š_2

Closed-Form Solution: wˆ* = (XˆT * X + * I)ˆ{-1} * XˆT * y

Optimization & Loss Function

Adds L2 penalty term * ||w||š to RSS, making (XˆT X + I) invertible even when features are correlated.

Meaning Breakdown & Mental Flowchart

residual sum squares

Minimize prediction error ||y - Xw||š

L2 penalty *||w||š

Shrink coefficient magnitudes towards zero

closed-form matrix

Solve wˆ* = (XˆT X + I)ˆ{-1} XˆT y in one step

46. LinearRegression

Mental Model

Find the optimal linear hyperplane that minimizes the sum of squared distances to all training data points.

Mathematical Formulation & Prediction

Prediction: y_hat = wˆT * x + b

Objective: min_{w} ||y - X w||š_2

Optimization & Loss Function

Closed-Form Ordinary Least Squares (OLS):
wˆ* = (XˆT * X)ˆ{-1} * XˆT * y

Computed via Singular Value Decomposition (SVD) of X.

Meaning Breakdown & Mental Flowchart

linear combination

Calculate y_hat = w_1 x_1 + ... + w_d x_d + b

squared residuals

Compute sum of squared errors (y_i - y_hat_i)š

SVD OLS solver

Find global minimum weights analytically via SVD

47. TransformedTargetRegressor

Mental Model

Transform non-linear target y (e.g., log transformation), train a regressor, and inverse-transform predictions back to real scale.

Mathematical Formulation & Prediction

Forward Target Transform: y_{trans} = g(y)

Fit: Regressor trained on (X, y_{trans})

Inverse Transform: y_pred = gˆ{-1}(f(x))

Optimization & Loss Function

Applies forward mapping g during fit() and inverse mapping gˆ{-1} during predict().

Meaning Breakdown & Mental Flowchart

transform y -> g(y)

Apply log / sqrt / custom transform to target y

fit base regressor

Train underlying regressor on transformed target

inverse gˆ{-1}(y_hat)

Apply inverse transformation to return predictions to real units

48. LassoCV

Mental Model

Compute the full Lasso L1 regularization path across an grid and pick the best using K-fold Cross-Validation.

Mathematical Formulation & Prediction

Lasso Path: w() computed via Coordinate Descent for _1 > _2 > ... > _K

Validation Score: ˆ* = argmin_{_k} (1/K) * _{v=1}ˆK MSE_{val}(_k, Fold_v)

Optimization & Loss Function

Computes entire Lasso regularization path efficiently along an grid using warm-restarted Coordinate Descent.

Meaning Breakdown & Mental Flowchart

grid path

Generate grid of regularization parameter values

coordinate descent

Compute sparse Lasso coefficient path w()

select optimal *

Pick * with minimal cross-validation MSE

49. ElasticNetCV

Mental Model

Cross-validate over a 2D grid of penalty scale and L1 ratio to find the optimal mix of Lasso and Ridge regularization.

Mathematical Formulation & Prediction

Objective: min_{w} (1/2n) ||y - X w||š + * [l1_ratio * ||w||_1 + 0.5 * (1 - l1_ratio) * ||w||š_2]

CV Selection: (ˆ*, l1_ratioˆ*) = argmin CV_MSE(, l1_ratio)

Optimization & Loss Function

Performs grid search over combinations of and l1_ratio via Coordinate Descent.

Meaning Breakdown & Mental Flowchart

2D parameter grid

Set candidate values for and l1_ratio

coordinate descent

Fit ElasticNet for each parameter pair across CV folds

optimal pair

Select optimal (*, l1_ratio*) with minimum validation error

50. LassoLarsCV

Mental Model

Compute the exact piecewise linear Lasso path using Least Angle Regression (LARS) and select via Cross-Validation.

Mathematical Formulation & Prediction

LARS Lasso Path: Computes exact piecewise linear breakpoints where features enter/exit the model.

Optimization & Loss Function

Evaluates cross-validation error along exact LARS breakpoints rather than a fixed heuristic grid.

Meaning Breakdown & Mental Flowchart

LARS path

Compute exact piecewise linear Lasso coefficient path

cross-validation

Evaluate prediction error at each path breakpoint

select best step

Pick optimal regularization step along LARS path

51. LassoLarsIC

Mental Model

Select the optimal Lasso parameter along the LARS path using Information Criteria (AIC or BIC) without cross-validation folds.

Mathematical Formulation & Prediction

AIC = n * log(MSE) + 2 * k

BIC = n * log(MSE) + k * log(n)

where k = ||w||_0 (number of non-zero active features)

Optimization & Loss Function

Selects optimal step along LARS path in a single training pass using analytical AIC or BIC score.

Meaning Breakdown & Mental Flowchart

LARS path

Generate exact piecewise linear feature entry path

compute AIC / BIC

Evaluate information criterion score for each path step

instant selection

Select optimal model without performing K-fold splits

52. LarsCV

Mental Model

Fit Least Angle Regression (LARS) and select the optimal number of steps using Cross-Validation.

Mathematical Formulation & Prediction

LARS Feature Direction: Move weights w in direction equiangular to features with highest correlation c_j = x_jˆT (y - y_hat).

Optimization & Loss Function

Evaluates cross-validation error along the LARS forward path to select optimal step count.

Meaning Breakdown & Mental Flowchart

equiangular path

Advance weights equiangularly as features enter active set

cross-validation

Measure out-of-fold MSE along LARS steps

select step count

Choose optimal number of features to retain

53. Lars

Mental Model

Stepwise forward feature selection algorithm that advances weight coefficients along equiangular correlation directions.

Mathematical Formulation & Prediction

Correlation Vector: c_j = x_jˆT * (y - y_hat_{k-1})

Equiangular Vector: Advance w in direction having equal correlation with all active features.

Optimization & Loss Function

Computes entire forward feature selection path in same order of computational complexity as a single OLS fit.

Meaning Breakdown & Mental Flowchart

start at w=0

Initialize all weights to zero

find max correlation

Select feature most correlated with current residual

move equiangularly

Advance weights until another feature reaches equal correlation

54. SGDRegressor

Mental Model

Optimize linear regression models iteratively using Stochastic Gradient Descent updates on mini-batches.

Mathematical Formulation & Prediction

Gradient Update: w_{t+1} = w_t - _t * [(w_tˆT x_i - y_i) * x_i + * R(w_t)]

Optimization & Loss Function

Loss Options: 'squared_error' (MSE), 'huber', 'epsilon_insensitive'. Regularization: L2, L1, or ElasticNet.

Meaning Breakdown & Mental Flowchart

mini-batch sample

Fetch random sample/batch from training data

calculate gradient

Compute loss gradient L with respect to weights w

update weights

Apply SGD step w_{t+1} = w_t - * L

55. RANSACRegressor

Mental Model

Fit a linear model on random subsets of data to separate true inlier points from extreme outlier noise.

Mathematical Formulation & Prediction

Inlier Condition: |y_i - f_{sub}(x_i)| t_{threshold}

Optimization & Loss Function

Iteratively fits base regressor on random sample subsets, counts consensus inliers within threshold t, and refits final model on the largest inlier set.

Meaning Breakdown & Mental Flowchart

random sample

Select minimum random data subset to fit trial model

count inliers

Identify all data points falling within residual threshold t

largest consensus

Find subset generating maximum inlier consensus set

refit final model

Re-estimate final regressor weights using inliers only

56. ElasticNet

Mental Model

Combine L1 (Lasso) feature selection with L2 (Ridge) grouping penalty to handle correlated features while maintaining sparsity.

Mathematical Formulation & Prediction

Objective:
min_{w} (1/2n) * ||y - X w||š_2 + * l1_ratio * ||w||_1 + 0.5 * * (1 - l1_ratio) * ||w||š_2

Optimization & Loss Function

Solved via Coordinate Descent using soft-thresholding updates.

Meaning Breakdown & Mental Flowchart

L1 penalty

Enforce sparsity by driving irrelevant feature weights to zero

L2 penalty

Encourage group selection among correlated feature variables

coordinate descent

Iteratively optimize each weight w_j while holding others fixed

57. Lasso

Mental Model

Apply an L1 absolute weight penalty to shrink unnecessary feature coefficients strictly to zero, performing automatic feature selection.

Mathematical Formulation & Prediction

Objective: min_{w} (1/2n) * ||y - X w||š_2 + * ||w||_1

Soft-Thresholding Update: w_j = S(X_jˆT (y - X_{-j} w_{-j}),) / ||X_j||š

Optimization & Loss Function

Solved via Coordinate Descent with Soft-Thresholding operator S(z,) = sgn(z) * max(0, |z| -).

Meaning Breakdown & Mental Flowchart

L1 penalty *||w||_1

Add sum of absolute weights to loss function

soft-thresholding

Pull small coefficients to exact zero w_j = 0

sparse feature set

Produce interpretable model with sparse non-zero coefficients

58. OrthogonalMatchingPursuitCV

Mental Model

Cross-validate over the number of non-zero features in Orthogonal Matching Pursuit to select optimal feature sparsity.

Mathematical Formulation & Prediction

OMP Path: Solves min ||y - X w||š s.t. ||w||_0 k for k = 1, 2, ...

CV Selection: kˆ* = argmin_k CrossValidation_MSE(k)

Optimization & Loss Function

Selects optimal sparsity parameter k using out-of-fold cross-validation error.

Meaning Breakdown & Mental Flowchart

OMP path

Generate greedy sparse feature selection path

cross-validation

Evaluate out-of-fold prediction MSE for feature counts k

select sparsity k*

Choose optimal non-zero feature count k*

59. ExtraTreeRegressor

Mental Model

Build a single extremely randomized regression tree using uniform random split thresholds.

Mathematical Formulation & Prediction

Split Threshold: t_j ~ Uniform(min(x_j), max(x_j))

Leaf Output: Average target value y_mean in leaf region

Optimization & Loss Function

Draws random thresholds per candidate feature to construct single fast randomized regression tree.

Meaning Breakdown & Mental Flowchart

random t_j

Draw random split threshold for feature candidate

variance reduction

Build fast randomized single regression tree

leaf mean

Output average target value of samples in leaf node

60. PassiveAggressiveRegressor

Mental Model

Stay passive if prediction error is within threshold ; aggressively update weights if error exceeds .

Mathematical Formulation & Prediction

Loss: L_ = max(0, |y_t - w_tˆT x_t| -)

Update: w_{t+1} = w_t + sgn(y_t - w_tˆT x_t) * _t * x_t

where _t = min(C, L_ / ||x_t||š)

Optimization & Loss Function

Online margin-based continuous regression update rule.

Meaning Breakdown & Mental Flowchart

error within

Passive mode: Keep current weights w_{t+1} = w_t

error exceeds

Aggressive mode: Update weights proportional to error loss

C bound

Clip step size _t by C to prevent noisy weight jumps

61. GaussianProcessRegressor

Mental Model

Model target values as a continuous Gaussian Process, providing non-parametric predictions with complete uncertainty bounds.

Mathematical Formulation & Prediction

Prior: f(x) ~ GP(m(x), k(x, x'))

Posterior Mean: ˆ* = K(xˆ*, X) * [K(X, X) + _nš I]ˆ{-1} * y

Posterior Variance: ˆ* = K(xˆ*, xˆ*) - K(xˆ*, X) * [K(X, X) + _nš I]ˆ{-1} * K(X, xˆ*)

Optimization & Loss Function

Maximizes Marginal Log-Likelihood to optimize kernel hyperparameters.

Meaning Breakdown & Mental Flowchart

kernel covariance K

Measure similarity between points via kernel k(x, x')

posterior mean *

Compute expected continuous target prediction

confidence bounds

Calculate variance * to output 95% uncertainty confidence interval

62. OrthogonalMatchingPursuit

Mental Model

Greedy forward feature selection algorithm that selects features most correlated with target residuals and projects orthogonally.

Mathematical Formulation & Prediction

Objective: min_{w} ||y - X w||š_2 s.t. ||w||_0 n_nonzero_coefs

Residual Update: r_k = y - X_{S_k} w_{S_k}

Optimization & Loss Function

At each step, picks feature x_j most correlated with current residual r_{k-1}, adds it to active set S_k, and projects target orthogonally onto X_{S_k}.

Meaning Breakdown & Mental Flowchart

pick max correlated x_j

Find feature most correlated with residual vector r

add to active set S

Include feature x_j in active feature subset

orthogonal projection

Project target y orthogonally onto selected features and update residual r

63. DecisionTreeRegressor

Mental Model

Recursively split feature space into rectangular regions and predict the mean target value of training samples in each leaf region.

Mathematical Formulation & Prediction

Node MSE: MSE = (1/N) * _{i Region} (y_i - y_mean)š

Split Optimization: min_{j, s} [MSE(Left) + MSE(Right)]

Optimization & Loss Function

Finds feature j and threshold s minimizing Mean Squared Error (or MAE) recursively until leaf stopping criteria are reached.

Meaning Breakdown & Mental Flowchart

test feature splits

Evaluate thresholds across features to minimize MSE

partition region

Split feature space into left and right sub-regions

leaf mean prediction

Predict average target y_mean for any sample landing in leaf

64. DummyRegressor

Mental Model

Make simple non-smart baseline predictions (like always predicting target mean or median) to benchmark real ML model performance.

Mathematical Formulation & Prediction

Strategy 'mean': y_pred = Mean(Y)

Strategy 'median': y_pred = Median(Y)

Strategy 'quantile': y_pred = Quantile(Y, q)

Optimization & Loss Function

Calculates simple summary statistic on training target values Y without inspecting input features X.

Meaning Breakdown & Mental Flowchart

ignore features X

Do not inspect input feature columns

calculate target summary

Compute mean or median of target column Y

output constant baseline

Return constant baseline prediction for all inputs

65. LassoLars

Mental Model

Solve Lasso L1 regularized linear regression along the exact piecewise linear Least Angle Regression (LARS) path.

Mathematical Formulation & Prediction

Objective: min_{w} (1/2n) * ||y - X w||š_2 + * ||w||_1

LARS Algorithm Variant: Modifies LARS to handle feature drops when coefficients hit zero.

Optimization & Loss Function

Computes exact Lasso regularization path in computationally efficient LARS steps.

Meaning Breakdown & Mental Flowchart

LARS path modification

Track features entering model and drop features hitting zero weight

piecewise linear w()

Compute exact analytical coefficient trajectory

exact Lasso weights

Return sparse weights w for target penalty

66. KernelRidge

Mental Model

Combine L2 regularized Ridge regression with non-linear kernel functions to fit smooth non-linear regression curves.

Mathematical Formulation & Prediction

Dual Objective: min_{} ||y - K ||š_2 + * ˆT * K *

Closed-Form Solution: ˆ* = (K + * I)ˆ{-1} * y

Prediction: f(x) = _{i=1}ˆn _i * K(x_i, x)

Optimization & Loss Function

Solves dual kernelized Ridge regression analytically via matrix inversion.

Meaning Breakdown & Mental Flowchart

kernel matrix K

Compute non-linear kernel matrix K_{ij} = K(x_i, x_j)

closed-form dual *

Solve ˆ* = (K + I)ˆ{-1} y in a single matrix inversion step

non-linear prediction

Evaluate smooth non-linear regression function f(x) = _i K(x_i, x)

Chapter 4: Computer Vision & Deep Learning Mental Models

4.1 Classic CNNs & Vision Transformers

Model / Architecture	Category	Mental Model & Core Idea	Primary Application
AlexNet	Classic CNN	Early deep CNN with ReLU and Dropout that revolutionized ImageNet classification.	Historical baseline CNN
VGG	Classic CNN	Very deep stack of simple 3x3 convolution filters and max pooling layers.	Classic feature extraction
ResNet	Residual CNN	Introduced skip connections (identity shortcuts) so 100+ layer networks train smoothly.	Standard visual backbone
DenseNet	Dense CNN	Connects every layer to all subsequent layers to maximize feature reuse.	Feature-reuse classification
Inception	Multi-Scale CNN	Runs multi-scale 1x1, 3x3, and 5x5 convolution filters in parallel in one block.	Multi-scale visual patterns
EfficientNet	Compounded CNN	Systematically scales depth, width, and resolution using compound coefficient.	High accuracy per compute
ConvNeXt	Modern CNN	Modernized CNN architecture incorporating design principles from Vision Transformers.	Modern state-of-art CNN
ViT (Vision Transformer)	Vision Transformer	Splits images into 16x16 patches and processes them as tokens via self-attention.	Global visual relationships
DeiT	Data-Efficient ViT	Uses teacher-student distillation to train Transformers efficiently on smaller data.	Data-efficient Transformers
Swin Transformer	Hierarchical ViT	Computes self-attention within shifted local windows for multi-scale dense prediction.	Object detection & segmentation
DINO / DINOv2 / DINOv3	Self-Supervised ViT	Self-distillation without labels producing rich generic visual features.	Foundation visual embeddings

4.2 Object Detection Architectures

Detector	Architecture Type	Mental Model & Mechanism	Primary Advantage
SSD	One-Stage CNN	Evaluates anchor boxes at multiple conv feature map resolutions in one pass.	Fast single-shot detection

Detector	Architecture Type	Mental Model & Mechanism	Primary Advantage
YOLO (v1-v10)	One-Stage CNN	Frames detection as a single regression grid problem for real-time inference.	Extreme speed & real-time video
Faster R-CNN	Two-Stage CNN	Uses Region Proposal Network (RPN) first, then crops and classifies candidate regions.	High spatial detection accuracy
RetinaNet	One-Stage CNN	Combines Feature Pyramid Network (FPN) with Focal Loss to handle severe background imbalance.	Accurate single-stage detection
FCOS	Anchor-Free CNN	Eliminates anchor boxes by predicting bounding box offsets directly at center points.	Anchor-free simplicity
DETR	Transformer Detector	Treats detection as a direct set prediction problem using Transformer encoder-decoder.	End-to-end NMS-free detection
Deformable DETR	Transformer Detector	Focuses self-attention on a small set of key sampling locations around reference points.	Fast multi-scale convergence
RT-DETR	Real-Time Transformer	First real-time end-to-end Transformer object detector balancing speed and accuracy.	Real-time Transformer detection
EfficientDet	Efficient Detector	Combines EfficientNet backbone with BiFPN multi-scale feature fusion.	Resource-efficient detection

4.3 Image Segmentation Architectures

Model	Segmentation Type	Mental Model & Mechanism	Best Used For
FCN	Semantic	Replaces fully-connected layers with deconvolutions for dense pixel classification.	First semantic segmentation
U-Net	Semantic	Symmetric U-shaped Encoder-Decoder with skip connections preserving spatial details.	Medical & small dataset masks
U-Net++	Semantic	Enhances U-Net with nested dense skip pathways to bridge semantic gap.	High-precision medical masks

Model	Segmentation Type	Mental Model & Mechanism	Best Used For
DeepLabV3+	Semantic	Uses Atrous (dilated) Spatial Pyramid Pooling (ASPP) to capture multi-scale context.	Complex urban scene segmentation
PSPNet	Semantic	Pyramid Scene Parsing network aggregating global context across spatial sub-regions.	Large context scene parsing
Mask R-CNN	Instance	Extends Faster R-CNN by adding a parallel branch for predicting pixel-level object masks.	Individual instance segmentation
YOLACT	Real-Time Instance	Generates prototype masks and predicts linear coefficients for rapid real-time masks.	Fast real-time instance masks
SegFormer	Semantic Transformer	Lightweight MLP decoder combined with hierarchical Transformer encoder.	Efficient Transformer masks

Chapter 5: Unsupervised Learning & Vector Mathematics

5.1 Distance & Similarity Metrics

Metric	Formula / Concept	Mental Model	Best Application
Cosine Similarity	cos() = (A ů B) / (||A|| ||B||)	Measures vector directional alignment	Text & visual vector search
Dot Product	A ů B = A_i * B_i	Measures magnitude and direction alignment	Dense embedding retrieval
Euclidean Distance	d = ((A_i - B_i)š)	Straight-line physical distance between points	Standard spatial distance
Manhattan Distance	d = |A_i - B_i|	Grid city-block distance along axes	High-dimensional sparse data
Minkowski Distance	d = (|A_i - B_i|ˆp)ˆ(1/p)	Generalized distance family (p=1 Manhattan, p=2 Euclidean)	Parametric distance optimization
Mahalanobis Distance	d = ((x -)ˆT ź (x -))	Distance accounting for feature correlations	Outlier detection in multivariate data
Chebyshev Distance	d = max_i |A_i - B_i|	Maximum absolute difference along any single axis	Chess king movement metric
Hamming Distance	d = I(A_i B_i)	Counts number of differing positions	Binary codes & string matching

5.2 Clustering Algorithms

Algorithm	Cluster Type	Mental Model & Mechanism	When to Use?
K-Means	Centroid-Based	Assigns points to nearest K centroids and updates centroids iteratively.	Known K, spherical clusters
Spherical K-Means	Directional Centroid	Cluster normalized vectors using cosine similarity.	Text & visual embeddings
DBSCAN	Density-Based	Grows clusters from dense neighborhoods; labels sparse points as noise.	Arbitrary shapes + noise filtering
HDBSCAN	Hierarchical Density	Extracts flat clusters from density hierarchy across varying densities.	Varying density, unknown K
OPTICS	Density Ordering	Orders points to analyze reachability plot across multiple density scales.	Complex density structures
Agglomerative	Hierarchical	Bottom-up merging of nearest clusters into a hierarchical tree (dendrogram).	Taxonomy & hierarchical groupings
Spectral Clustering	Graph-Based	Computes Laplacian matrix eigenvectors and clusters in low-dimensional space.	Complex non-convex cluster shapes
Mean Shift	Centroid Shift	Shifts points towards local modes of kernel density estimate.	Blob detection, unknown K

Algorithm	Cluster Type	Mental Model & Mechanism	When to Use?
GMM	Probabilistic	Fits mixture of K Multivariate Gaussians; provides soft probability membership.	Overlapping soft clusters

5.3 Dimensionality Reduction Techniques

Technique	Type	Mental Model & Mechanism	Primary Purpose
PCA	Linear Unsupervised	Rotates coordinate axes to directions of maximum variance.	Compression & preprocessing
LDA	Linear Supervised	Projects data to maximize between-class over within-class variance.	Supervised dimensionality reduction
t-SNE	Non-linear Manifold	Converts distances to probabilities and preserves local neighbor structures.	2D/3D visual cluster exploration
UMAP	Non-linear Manifold	Uses Riemannian geometry to preserve local + global manifold structure.	Fast 2D/3D visualization & ML input
Kernel PCA	Non-linear Kernel	Applies Kernel trick K(x, z) before computing principal components.	Non-linear feature extraction
Truncated SVD	Linear Matrix	Performs SVD without centering data; works on sparse matrices.	Text TF-IDF & sparse matrices
Autoencoder	Neural Network	Encoder squeezes input into bottleneck code; decoder reconstructs.	Deep non-linear feature compression

Chapter 6: Time Series Analysis & Forecasting

6.1 Time Series Fundamentals

A time series is a sequence of observations indexed by time. Unlike ordinary tabular data, time-series observations are usually dependent on previous observations, making temporal ordering critical.

Concept	Definition / Formula	Mental Model	Purpose
Time Series	y_t observed at time t	Ordered observations through time	Forecasting and temporal analysis
Trend	Long-term systematic movement in y_t	Overall upward or downward direction	Identify long-term growth or decline
Seasonality	Repeating pattern with fixed period	Pattern repeats every day, week, month, etc.	Capture calendar-based effects
Cycle	Long-term fluctuation without fixed period	Business/economic waves	Model non-fixed periodic behavior
Noise	Random unpredictable component	Irregular variation	Quantify unexplained variation
Level	Baseline value around which observations fluctuate	Current underlying magnitude	Foundation for forecasting
Lag	y_{t-k}	Value from k time steps ago	Represent temporal dependence
Autocorrelation	$\rho_k = \text{Corr}(y_t, y_{t-k})$	Similarity with past values	Detect temporal dependence
Stationarity	Statistical properties remain stable over time	Same distributional behavior through time	Required by many classical models
White Noise	$\mathbb{E}[\epsilon_t] = 0$, constant variance, no autocorrelation	Pure random error	Desired residual behavior
Random Walk	$y_t = y_{t-1} + \epsilon_t$	Current value = previous value + shock	Baseline non-stationary process

6.2 Components of a Time Series

A time series can often be represented using additive or multiplicative decomposition.

Component	Concept	Mental Model	Example
Trend	Long-term movement	General direction	Increasing sales
Seasonality	Repeating fixed-period pattern	Calendar effect	Higher retail sales every December
Cycle	Non-fixed long-term fluctuation	Economic/business cycle	Recession and recovery
Residual	Remaining unexplained component	Noise after modeling structure	Measurement randomness

Additive decomposition:

$$y_t = T_t + S_t + R_t$$

where T_t is trend, S_t is seasonality, and R_t is the residual.

Multiplicative decomposition:

$$y_t = T_t \times S_t \times R_t$$

Use additive decomposition when seasonal variation is approximately constant and multiplicative decomposition when seasonal variation scales with the level of the series.

6.3 Time Series Preprocessing

Technique	Concept	Mental Model	Purpose
Datetime Parsing	Convert timestamps into datetime objects	Make time machine-readable	Correct temporal processing
Sorting	Sort observations chronologically	Put history in correct order	Prevent temporal leakage
Resampling	Aggregate observations to another frequency	Change time resolution	Daily to weekly/monthly data
Missing Value Handling	Imputation, interpolation, forward/backward fill	Repair missing history	Produce continuous series
Outlier Detection	Identify unusually large deviations	Find abnormal observations	Improve model robustness
Rolling Statistics	Mean, variance, etc. over moving windows	Local summary of recent history	Detect trend and changing variance
Lag Features	$y_{t-1}, y_{t-2}, \dots$	Give model historical context	Machine-learning forecasting
Calendar Features	Hour, day, week, month, holiday, etc.	Encode temporal context	Capture systematic patterns
Scaling	Standardization or normalization	Put numerical features on comparable scales	Useful for ML and neural networks

6.4 Stationarity

A stationary time series has statistical properties that do not systematically change over time. In weak stationarity, the mean and variance are constant and covariance depends primarily on the lag rather than the absolute time:

$$\mathbb{E}[y_t] = \mu$$

$$\text{Var}(y_t) = \sigma^2$$

$$\text{Cov}(y_t, y_{t-k}) = \gamma_k$$

Stationarity is especially important for ARIMA-type models.

Differencing

First-order differencing is defined as:

$$\Delta y_t = y_t - y_{t-1}$$

Second-order differencing is:

$$\Delta^2 y_t = \Delta y_t - \Delta y_{t-1}$$

Differencing removes trend-like non-stationarity.

Seasonal Differencing

For seasonal period s:

$$\Delta_s y_t = y_t - y_{t-s}$$

This is useful when a series contains repeating seasonal patterns.

6.5 Stationarity Tests

Test	Core Idea	Null Hypothesis	Interpretation
ADF Test	Tests for unit root	Series has a unit root	Small p-value suggests stationarity
KPSS Test	Tests stationarity around level/trend	Series is stationary	Large p-value supports stationarity
Phillips-Perron	Unit-root test robust to serial correlation	Unit root exists	Small p-value supports stationarity

ADF and KPSS are often used together because their null hypotheses are different.

6.6 Autocorrelation and Partial Autocorrelation

Autocorrelation Function (ACF) measures correlation between a series and its lagged versions:

$$\rho_k = \frac{\text{Cov}(y_t, y_{t-k})}{\sqrt{\text{Var}(y_t)\text{Var}(y_{t-k})}}$$

Partial Autocorrelation Function (PACF) measures the correlation at lag k after removing the effects of intermediate lags.

Tool	Measures	Mental Model	Main Use
ACF	Correlation with previous lags	Direct temporal dependence	Identify MA order and seasonality
PACF	Direct correlation after intermediate lags are removed	Isolate direct lag effect	Identify AR order

6.7 Classical Forecasting Methods

Method	Formula / Mechanism	Mental Model	When to Use
Naive Forecast	$\hat{y}_{t+1} = y_t$	Tomorrow equals today	Strong baseline
Seasonal Naive	$\hat{y}_{t+h} = y_{t+h-s}$	Repeat previous season	Strong seasonality
Mean Forecast	$\hat{y}_{t+h} = \bar{y}$	Predict historical average	Stable series
Drift Method	Extends average historical trend	Continue overall direction	Trending series
Moving Average	$\hat{y}_t = \frac{1}{k}\sum_{i=1}^{k} y_{t-i}$	Average recent history	Smooth short-term noise
Weighted Moving Average	Weighted recent observations	Recent values matter more	Changing local level
Simple Exponential Smoothing	$\hat{y}_{t+1} = \alpha y_t + (1-\alpha)\hat{y}_t$	Weighted memory of past	Series with level but no strong trend
Holt's Method	Models level and trend	Exponential smoothing with trend	Trending data

Method	Formula / Mechanism	Mental Model	When to Use
Holt-Winters	Models level, trend and seasonality	Exponential smoothing with seasonality	Seasonal forecasting

6.8 Autoregressive Models

An autoregressive model predicts the current value using previous observations:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \epsilon_t$$

This is an AR(p) model.

Model	Mechanism	Mental Model	Use
AR(1)	Uses one previous value	Depend on yesterday	Simple temporal dependence
AR(p)	Uses p previous values	Memory of recent history	Autoregressive forecasting

6.9 Moving Average Models

A Moving Average model predicts the current observation using previous error terms rather than previous observations:

$$y_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

This is an MA(q) model.

6.10 ARMA

ARMA combines autoregressive and moving-average components:

$$y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + \epsilon_t + \sum_{j=1}^{q} \theta_j \epsilon_{t-j}$$

Model	Components	Mental Model	Requirement
AR	Past observations	Memory of values	Stationary series
MA	Past errors	Memory of shocks	Stationary series
ARMA	Past observations + past errors	Combined memory	Stationary series

6.11 ARIMA

ARIMA stands for Autoregressive Integrated Moving Average:

$$\text{ARIMA}(p, d, q)$$

where:

- p = autoregressive order,
- d = number of differences,
- q = moving-average order.

The model first differences the series d times to achieve stationarity and then applies AR and MA components.

Parameter	Meaning	Typical Diagnostic	Mental Model
p	AR order	PACF	How many past values matter
d	Differencing order	Stationarity tests	How much trend must be removed
q	MA order	ACF	How many past errors matter

6.12 Seasonal ARIMA (SARIMA)

SARIMA extends ARIMA by explicitly modeling seasonal behavior:

$$\text{SARIMA}(p, d, q)(P, D, Q)_s$$

where P, D, and Q are seasonal AR, differencing, and MA orders, and s is the seasonal period.

Parameter	Meaning	Example	Purpose
p	Non-seasonal AR order	2	Recent observations
d	Non-seasonal differencing	1	Remove trend
q	Non-seasonal MA order	1	Past shocks
P	Seasonal AR order	1	Seasonal memory
D	Seasonal differencing	1	Remove seasonal non-stationarity
Q	Seasonal MA order	1	Seasonal shocks
s	Seasonal period	12 for monthly data	Define season length

6.13 Vector Autoregression (VAR)

VAR models multiple interacting time series simultaneously:

$$\mathbf{y}_t = \mathbf{c} + A_1\mathbf{y}_{t-1} + A_2\mathbf{y}_{t-2} + \cdots + A_p\mathbf{y}_{t-p} + \boldsymbol{\epsilon}_t$$

Model	Concept	Mental Model	Use
VAR	Multiple time series predict each other	Interacting temporal system	Multivariate forecasting
VARMA	VAR + moving-average errors	Multivariate ARMA	Complex multivariate series
VARMAX	VAR with exogenous variables	Multiple series + external drivers	Forecasting with additional predictors

6.14 Exponential Smoothing Family

Method	Components	Mental Model	Use
SES	Level	Recent observations weighted more heavily	No trend or seasonality
Holt	Level + trend	Exponentially weighted trend	Trending series
Holt-Winters Additive	Level + trend + additive seasonality	Constant seasonal amplitude	Additive seasonality

Method	Components	Mental Model	Use
Holt-Winters Multiplicative	Level + trend + multiplicative seasonality	Seasonal amplitude scales with level	Multiplicative seasonality

6.15 State Space Models

State space models represent a time series using hidden states that evolve through time:

$$\mathbf{x}_t = F\mathbf{x}_{t-1} + \mathbf{w}_t$$

$$\mathbf{y}_t = H\mathbf{x}_t + \mathbf{v}_t$$

where $\mathbf{x}_t$ is the latent state, $\mathbf{y}_t$ is the observed measurement, and $\mathbf{w}_t, \mathbf{v}_t$ represent process and observation noise.

Technique	Concept	Mental Model	Application
State Space Model	Hidden state + observation equation	Hidden system generates observations	Dynamic systems
Kalman Filter	Recursive state estimation	Continuously update belief from new observations	Tracking and filtering
Kalman Smoother	Uses future observations to refine past states	Look backward with full information	State estimation

6.16 Prophet-Style Decomposition Models

Modern additive forecasting frameworks decompose a series into interpretable components:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

where:

- $g(t)$ represents trend,
- $s(t)$ represents seasonality,
- $h(t)$ represents holiday or event effects,
- ϵ_t represents noise.

These models are particularly useful for business time series containing multiple seasonalities, trend changes, holidays, and missing observations.

6.17 Fourier Features

Periodic patterns can be represented using sine and cosine functions:

$$y_t = \beta_0 + \sum_{k=1}^{K} \left[a_k \sin\left(\frac{2\pi k t}{m}\right) + b_k \cos\left(\frac{2\pi k t}{m}\right) \right] + \epsilon_t$$

where m is the seasonal period and K controls the number of Fourier terms.

Technique	Concept	Mental Model	Use
Fourier Terms	Sine/cosine basis functions	Encode repeating cycles	Multiple or long seasonal periods
Seasonal Harmonics	Multiple Fourier frequencies	Capture complex seasonal shape	Smooth seasonal patterns

6.18 Machine Learning for Time Series

Time series can be converted into supervised-learning problems using lag, rolling, calendar, and external features.

Algorithm	Mechanism	Mental Model	When to Use
Linear Regression	Predicts target from lag and external features	Weighted combination of predictors	Simple interpretable forecasting
Ridge Regression	Linear regression + L2 regularization	Shrinks correlated coefficients	Many correlated lag features
Lasso Regression	Linear regression + L1 regularization	Selects useful features	Sparse feature selection
Elastic Net	L1 + L2 regularization	Combination of shrinkage and selection	High-dimensional lag features
Random Forest	Ensemble of decision trees	Many trees vote together	Non-linear forecasting relationships
Gradient Boosting	Sequentially improves weak learners	Each tree fixes previous errors	Strong tabular forecasting baseline
XGBoost	Optimized gradient boosting	Regularized high-performance boosting	Production forecasting with engineered features
LightGBM	Histogram-based gradient boosting	Fast tree boosting	Large-scale forecasting
CatBoost	Ordered boosting with categorical handling	Robust boosting with categorical features	Time series with categorical variables

6.19 Deep Learning for Time Series

Architecture	Mechanism	Mental Model	Application
MLP	Feed-forward network over engineered temporal features	Non-linear feature mapping	Basic neural forecasting
RNN	Recurrently processes sequential observations	Hidden memory state	Sequential dependencies
LSTM	RNN with gated memory cells	Long-term memory with controlled forgetting	Long temporal dependencies
GRU	Simplified gated recurrent network	Compact LSTM alternative	Efficient sequence modeling
1D CNN	Convolution over temporal windows	Detect local temporal patterns	Signal and sequence forecasting
TCN	Dilated causal convolutions	Large temporal receptive field without recurrence	Long sequence modeling
Seq2Seq	Encoder-decoder architecture	Encode history and generate future sequence	Multi-step forecasting
Transformer	Self-attention over sequence positions	Learn relationships between time steps	Long-range temporal dependencies

Architecture	Mechanism	Mental Model	Application
Temporal Fusion Transformer	Attention + variable selection + recurrent/temporal components	Interpretable multi-horizon forecasting	Complex multivariate forecasting
N-BEATS	Deep residual blocks specialized for forecasting	Learn basis functions for trend/seasonality	Univariate forecasting
N-HiTS	Hierarchical interpolation for forecasting	Predict multiple temporal resolutions	Long-horizon forecasting

6.20 Attention and Transformers for Time Series

Self-attention is based on:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, and V represent query, key, and value matrices.

Concept	Mechanism	Mental Model	Purpose
Self-Attention	Each time step attends to other time steps	Learn temporal relationships	Long-range dependencies
Multi-Head Attention	Multiple attention projections	Several temporal relationships simultaneously	Rich temporal representations
Causal Attention	Prevents access to future observations	Only use available history	Autoregressive forecasting
Positional Encoding	Adds temporal position information	Tell model where a value occurs	Preserve temporal order

6.21 Multivariate Time Series

A multivariate time series contains multiple variables observed over time.

Concept	Definition	Mental Model	Use
Univariate	One variable through time	One signal	Single-series forecasting
Multivariate	Multiple variables through time	Interacting signals	Complex forecasting
Exogenous Variable	External predictor X_t	Outside information influencing target	Weather, price, promotions
Cross-Correlation	Correlation between different series at different lags	How one series relates to another over time	Feature selection and causality analysis

6.22 Granger Causality

Granger causality tests whether historical values of one variable provide useful predictive information for another variable.

If X helps predict Y beyond the information contained in the historical values of Y, then X is said to Granger-cause Y:

$$Y_t = \sum_i a_i Y_{t-i} + \sum_j b_j X_{t-j} + \epsilon_t$$

A significant contribution from the X terms suggests predictive causality, but it should not be interpreted as proof of real-world causal mechanisms.

6.23 Cointegration

Two non-stationary series may have a stationary linear combination. Such series are said to be cointegrated:

$$z_t = y_t - \beta x_t$$

If y_t and x_t are individually non-stationary but z_t is stationary, the variables may share a long-run equilibrium relationship.

Technique	Concept	Mental Model	Use
Engle-Granger Test	Tests residual-based cointegration	Check whether combination becomes stationary	Two-series cointegration
Johansen Test	Multivariate cointegration test	Find number of cointegrating relationships	Multiple time series
VECM	Vector Error Correction Model	Short-run dynamics + long-run equilibrium	Cointegrated multivariate series

6.24 Time Series Cross-Validation

Random train-test splitting is generally inappropriate for forecasting because it can allow information from the future to enter the training process.

Method	Mechanism	Mental Model	Use
Holdout Split	Earlier observations train, later observations test	Simulate future prediction	Basic evaluation
Expanding Window	Training set continually grows	Learn from all available history	Forecasting validation
Rolling Window	Training window moves forward	Fixed amount of recent history	Non-stationary environments
Walk-Forward Validation	Train, predict next period, move forward	Repeatedly simulate deployment	Realistic forecasting evaluation
Blocked Time Series CV	Sequential non-overlapping validation blocks	Preserve temporal ordering	Robust model comparison

6.25 Forecasting Horizons

Type	Definition	Example	Challenge
One-Step Forecast	Predict next time step	Tomorrow's demand	Short-term uncertainty
Multi-Step Recursive	Predict one step repeatedly using predictions	Forecast next 30 days	Errors accumulate
Direct Multi-Step	Separate model for each horizon	One model per future day	More models to train
Multi-Output	One model predicts all future steps	Predict entire horizon simultaneously	More complex output

6.26 Forecast Uncertainty and Prediction Intervals

A point forecast provides a single expected value, while a prediction interval provides a range representing forecast uncertainty:

$$\hat{y}_{t+h} \pm z_{\alpha/2}\sigma_h$$

where σ_h is the forecast standard deviation at horizon h.

Uncertainty generally increases as the forecast horizon becomes longer.

6.27 Forecast Evaluation Metrics

Metric	Formula	Mental Model	Best Use
MAE	$\frac{1}{n}\sum \lvert y_i - \hat{y}_i \rvert$	Average absolute error	Robust, interpretable error
MSE	$\frac{1}{n}\sum (y_i - \hat{y}_i)^2$	Penalizes large errors	Optimization
RMSE	$\sqrt{\text{MSE}}$	Error in original units	Forecast comparison
MAPE	$\frac{100}{n}\sum \left\lvert \frac{y_i - \hat{y}_i}{y_i} \right\rvert$	Percentage error	Scale-independent evaluation
sMAPE	$\frac{100\%}{n}\sum \frac{\lvert y_i - \hat{y}_i \rvert}{(\lvert y_i \rvert + \lvert \hat{y}_i \rvert)/2}$	Symmetric percentage error	Comparing different scales
WAPE	$\frac{\sum \lvert y_i - \hat{y}_i \rvert}{\sum \lvert y_i \rvert}$	Weighted absolute percentage error	Demand forecasting
MASE	$\frac{\text{MAE}}{\text{MAE}_{\text{naive}}}$	Scaled MAE relative to naive forecast	Cross-series comparison
RMSLE	$\sqrt{\frac{1}{n}\sum (\log(1+\hat{y}_i) - \log(1+y_i))^2}$	Penalizes relative differences	Positive skewed targets

6.28 Residual Analysis

A good forecasting model should leave residuals that resemble white noise:

$$e_t = y_t - \hat{y}_t$$

Diagnostic	What to Check	Desired Result	Problem Indicated
Residual Mean	Average error	Approximately zero	Systematic bias
Residual Variance	Error spread	Approximately constant	Heteroscedasticity
Residual ACF	Autocorrelation	No significant correlation	Uncaptured temporal structure
Residual Distribution	Shape of errors	Approximately well-behaved	Non-Gaussian errors may affect inference
Ljung-Box Test	Joint autocorrelation test	Large p-value	Significant residual autocorrelation if rejected

6.29 Heteroscedasticity and Volatility Models

When the variance of a time series changes over time, volatility models can be used.

Model	Mechanism	Mental Model	Application
ARCH	Models conditional variance using past squared errors	Volatility depends on previous shocks	Financial returns
GARCH	Models variance using past shocks and past variance	Volatility has memory	Financial volatility

Model	Mechanism	Mental Model	Application
EGARCH	Log-volatility model	Handles asymmetric volatility effects	Leverage effects
GJR-GARCH	Adds asymmetric shock response	Negative and positive shocks can differ	Equity volatility

A basic GARCH(p, q) model can be represented as:

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2$$

6.30 Change Point Detection

Change point detection identifies times when the statistical behavior of a series changes.

Method	Concept	Mental Model	Use
CUSUM	Accumulates deviations from expected behavior	Detect persistent shifts	Process monitoring
PELT	Penalized optimization for multiple change points	Efficient segmentation	Large time series
Binary Segmentation	Recursively splits series at detected changes	Divide and conquer	Multiple regime changes
Bayesian Change Point	Estimates probability of changes	Probabilistic regime detection	Uncertainty-aware analysis

6.31 Anomaly Detection in Time Series

Method	Mechanism	Mental Model	Application
Z-Score	Measures standard deviations from mean	How unusual is this value?	Simple anomaly detection
IQR Method	Uses quartile-based thresholds	Detect extreme observations	Robust outlier detection
Isolation Forest	Randomly isolates unusual observations	Anomalies are easier to isolate	Multivariate anomaly detection
One-Class SVM	Learns boundary around normal observations	Separate normal from abnormal	Novelty detection
Autoencoder	Learns to reconstruct normal patterns	Poor reconstruction indicates anomaly	Complex temporal anomalies
Forecast-Based Detection	Flag unusually large forecast residuals	Unexpected relative to predicted value	Operational monitoring

6.32 Frequency-Domain Analysis

Time-domain analysis studies values over time, while frequency-domain analysis studies repeating frequencies within the signal.

Technique	Concept	Mental Model	Purpose
Fourier Transform	Converts time-domain signal into frequency components	Find hidden cycles	Periodicity analysis

Technique	Concept	Mental Model	Purpose
FFT	Fast algorithm for Fourier Transform	Efficient frequency decomposition	Signal processing
Periodogram	Estimates power across frequencies	Which frequencies contain energy?	Detect dominant cycles
Spectral Density	Distribution of signal power over frequency	Frequency fingerprint	Signal characterization
Wavelet Transform	Localized time-frequency representation	Analyze frequency that changes through time	Non-stationary signals

6.33 Causality and Temporal Relationships

Technique	Concept	Mental Model	Purpose
Cross-Correlation	Correlation between two series at different lags	Find delayed relationships	Lag discovery
Granger Causality	Tests predictive contribution of another series	Does past X improve prediction of Y?	Predictive causality
Impulse Response	Tracks response of a system to a shock	What happens after a sudden change?	VAR interpretation
Forecast Error Variance Decomposition	Attributes forecast variance to different shocks	Which variable explains uncertainty?	Multivariate interpretation

6.34 Time Series Feature Engineering

Feature	Example	Mental Model	Purpose
Lag Features	$y_{t-1}, y_{t-7}, y_{t-30}$	Historical memory	Autoregressive information
Rolling Mean	mean$(y_{t-k:t})$	Recent local level	Smooth signal
Rolling Std	std$(y_{t-k:t})$	Recent volatility	Capture changing variance
Rolling Min/Max	Minimum or maximum over window	Recent extremes	Demand and risk features
Expanding Mean	Mean from beginning to current point	Long-term average	Historical baseline
Difference Features	$y_t - y_{t-k}$	Recent change	Momentum and trend
Percentage Change	$\frac{y_t - y_{t-k}}{y_{t-k}}$	Relative change	Growth rate
Calendar Features	Hour, weekday, month, quarter	Temporal context	Seasonality
Holiday Features	Holiday indicator	Event effect	Special-day patterns
Fourier Features	Sine/cosine seasonal terms	Smooth periodic encoding	Long seasonal patterns
External Features	Weather, price, promotions	Outside drivers	Causal/predictive context

6.35 Time Series Leakage

Temporal leakage occurs when information that would not have been available at prediction time is used during training.

Common examples include:

- Randomly shuffling time-series observations before splitting.
- Computing rolling statistics using future observations.
- Scaling the complete dataset before creating a chronological split.

- Using future target values to construct features.
- Performing interpolation using observations from the future.

The fundamental rule is:

Features at time t must only use information available at or before t

6.36 Forecasting Strategy Selection

Data Situation	Recommended Models	Mental Model	Priority
Stable level	Naive, SES	Recent level predicts future	Simple baseline
Strong trend	Holt, ARIMA, Gradient Boosting	Continue learned trend	Trend modeling
Strong seasonality	Holt-Winters, SARIMA, Fourier features	Repeat seasonal structure	Seasonal forecasting
Many external variables	XGBoost, LightGBM, CatBoost, TFT	Learn nonlinear relationships	Feature-rich forecasting
Long temporal dependencies	LSTM, TCN, Transformer	Learn long memory	Deep sequence modeling
Multiple interacting series	VAR, VECM, TFT, Transformers	Variables influence each other	Multivariate forecasting
Changing volatility	ARCH, GARCH	Model conditional variance	Financial forecasting
Unknown structural changes	Change-point models, HDBSCAN-style segmentation, state-space models	Detect regimes	Regime-aware forecasting
Small dataset	Naive, ETS, ARIMA, SARIMA	Statistical assumptions help	Classical forecasting
Large dataset	Boosting, deep learning, Transformers	Learn complex patterns	Scalable ML forecasting

6.37 End-to-End Time Series Workflow

A practical forecasting pipeline can be summarized as:

Collect $\rightarrow$ Sort $\rightarrow$ Clean $\rightarrow$ Explore $\rightarrow$ Decompose $\rightarrow$ Test Stationarity $\rightarrow$ Feature Engineer $\rightarrow$ Split Chronologically $\rightarrow$ Baseline $\rightarrow$ Train $\rightarrow$ Validate $\rightarrow$ Diagnose $\rightarrow$ Forecast

Stage	Key Activities	Main Tools	Goal
1. Data Preparation	Parse timestamps, sort, handle missing values	Pandas, Polars	Clean chronological dataset
2. Exploration	Plot series, rolling statistics, ACF/PACF	Visualization, statistics	Understand temporal structure
3. Decomposition	Trend, seasonality, residual analysis	STL, classical decomposition	Separate components
4. Stationarity	ADF, KPSS, differencing	Statistical tests	Determine integration requirements
5. Feature Engineering	Lags, rolling statistics, calendar and Fourier features	Pandas, feature pipelines	Create predictive information

Stage	Key Activities	Main Tools	Goal
6. Baseline	Naive, seasonal naive, moving average	Simple forecasting models	Establish benchmark
7. Model Training	ARIMA, SARIMA, ETS, ML, deep learning	Statistical/ML frameworks	Learn forecasting relationship
8. Validation	Walk-forward or expanding-window validation	Time-series CV	Estimate future performance
9. Diagnostics	Residual ACF, Ljung-Box, error analysis	Statistical diagnostics	Verify model assumptions
10. Forecasting	Point forecasts and prediction intervals	Selected model	Predict future values
11. Monitoring	Track errors, drift and anomalies	Monitoring pipeline	Maintain production accuracy

6.38 Quick Algorithm Selection Guide

Question	Preferred Starting Point	Reason
Do I need a simple baseline?	Naive / Seasonal Naive	Extremely difficult to beat on some series
Is there trend but little seasonality?	Holt / ARIMA	Explicit trend modeling
Is there strong seasonality?	Holt-Winters / SARIMA	Explicit seasonal structure
Is the data non-stationary?	Differencing / ARIMA / SARIMA	Remove temporal non-stationarity
Are there many external features?	Gradient Boosting	Strong nonlinear tabular learner
Are there many correlated series?	VAR / VECM / multivariate neural models	Model cross-series relationships
Do I need probabilistic forecasts?	ETS / State Space / probabilistic neural models	Produce uncertainty estimates
Are there structural breaks?	Change-point / state-space methods	Explicitly model regime changes
Is the dataset very large?	Boosting / deep learning / Transformers	Scale to complex datasets
Do I need interpretability?	Naive / ETS / ARIMA / linear regression	Transparent model structure
Do I need long-horizon deep forecasting?	TFT / N-BEATS / N-HiTS / Transformers	Designed for multi-horizon prediction

6.39 Key Time Series Concepts to Remember

- **Trend** describes long-term direction.
- **Seasonality** describes repeating fixed-period behavior.
- **Stationarity** means the statistical behavior of the series is stable through time.
- **ACF** measures correlation with lagged observations.
- **PACF** measures direct lag relationships after accounting for intermediate lags.
- **AR** models depend on previous observations.
- **MA** models depend on previous forecast errors.
- **ARIMA** combines autoregression, differencing, and moving average.
- **SARIMA** adds explicit seasonal components to ARIMA.
- **ETS/Holt-Winters** models level, trend, and seasonality through exponential smoothing.

- **VAR** models multiple interacting time series.
- **GARCH** models changing conditional variance.
- **Granger causality** concerns predictive usefulness, not proof of true causation.
- **Cointegration** describes long-run equilibrium relationships between non-stationary series.
- **Walk-forward validation** preserves the temporal structure of the forecasting problem.
- **Data leakage** occurs when future information enters the training process.
- **Residuals should ideally resemble white noise** after a good forecasting model has captured the systematic structure.